COMPUTER ENGINEERING

QUEZON CITY

COURSE CODE/TITLE:	CPE010	**SECTION:**	CPE21S4
DATE PERFORMED:	09/16/2026	**DATE SUBMITTED:**	09/16/2026
NAME:	JAMES DANIEL M. VERANO	**INSTRUCTOR:**	ENGR. JIMLORD QUEJADO

ACTIVITY NO. 8.1 **Sorting Algorithm P2**

1. PROCEDURE OUTPUT

Code + Console Screenshot

```
const int maxSize = 100;
void displayArr(int arr[]);

int main(){

    //create an array with unsorted list
    int unsortedArr[maxSize];
    // use time (0) as seed for random
    srand(time(0));
    //populate the array with random val
    for(int i = 0; i < maxSize; i++){
        unsortedArr[i] = rand()%100;
    }
```

```
void displayArr(int arr[]){
    for(int i = 0; i < maxSize; i++){
        std::cout << arr[i] << " ";
        if ((i%10) == 9){
            std::cout << '\n';
        }
    }
}
```

```
Unsorted Array:
72 65 13 83 73 86 82 36 76 8
65 2 44 22 49 51 76 33 16 79
70 58 20 64 47 85 2 42 85 21
59 26 5 77 40 85 89 66 68 39
44 70 48 20 5 77 26 85 86 97
32 55 99 83 3 26 30 7 11 60
79 21 46 68 50 42 61 33 24 77
54 46 96 39 22 64 3 99 88 9
98 2 7 63 26 73 41 4 55 93
36 76 22 92 5 17 49 68 35 1
```

Observations

First is to create an array of elements with random values and the array must contain a max

	element of 100 values. It is stated that it should not be sorted, Therefore, the algorithm for the creation for the printing of array of values is, #A max size array of 100 #A for loop that will generate 100 random numbers #printing loop should be integrated inside a function #print the 100 data in the main.cpp file by function. Which is now represented as the code implementation above. First is the constant declaration for the **max_size array of 100 random values**. Next is the declaration of the **function prototype for the needed parameter** of the function, which is the function that is the generating of 100 types of random values using the **rand()** integrated with **srand(time)** for the processing of generating random values. Once data is generated, the displayAll function is called for the printing of the 100 random values.

Table 8-1. Array of Values For Sort Algorithm Testing

Code + Console Screenshot

```cpp
template<typename T>
void shellSort(T array[],int size){
    // Loops while the comparison gap is greater than 0
    for (int gap = size/2;gap > 0; gap /=2){
        //selection like sorting
        for (int i = gap;i < size;i++){
            int temp = array[i];
            int key = i;
            //Continues shifting elements forward as the front of the gap chain
            // has not reached the end, and the previous gap element is larger than temp
            while (key >= gap && array[key - gap] > temp) {
                array[key] = array[key - gap];
                key -= gap;
            }
            array[key] = temp;
        }
    }
}
```

```
Unsorted Array:
11 25 17 71 78 15 50 90 15 19
21 22 3 88 2 92 20 24 90 40
3 31 7 75 59 76 43 21 4 3
38 49 0 51 19 91 29 61 27 58
83 28 68 74 11 2 54 19 87 63
35 6 28 61 12 91 66 24 74 28
23 11 50 41 39 9 53 29 54 49
63 40 11 48 98 53 63 21 65 22
33 79 93 45 68 73 36 1 69 2
36 46 20 30 14 89 61 53 12 28

Sorted Array Using Shell Sort:
 0 1 2 2 2 3 3 3 4 6
7 9 11 11 11 11 12 12 14 15
15 17 19 19 19 20 20 21 21 21
22 22 23 24 24 25 27 28 28 28
28 29 29 30 31 33 35 36 36 38
39 40 40 41 43 45 46 48 49 49
50 50 51 53 53 53 54 54 58 59
61 61 61 63 63 63 65 66 68 68
69 71 73 74 74 75 76 78 79 83
87 88 89 90 90 91 91 92 93 98
```

Observations

In this shell sort, the function has an adapted variant of insertion sort. The difference is that, instead of moving elements one position at a time, it divides the whole array using the gap variable by 2. The purpose of this is that it contains a nested loop condition where it terminates early when elements are already in relative order. It compares the 2 divided arrays to one another and sorts them. This sorting algorithm has a memory efficiency of O(1) auxiliary space as it rearranges data directly with native array parameters using

	standard variable swappings.

Table 8-2. Shell Sort Technique

Code + Console Screenshot	

```
template<typename T>
void merge(T array[],int left,int mid,int right){
    // A helper function that takes two adjacent sorted halves of an array
    int sizeL = mid-left + 1; // Calculation for the left partition
    int sizeR = right - mid; // Calculation for the right partition

    // temporary array for the left and right half
    T arrL[sizeL];
    T arrR[sizeR];

    for (int i = 0; i < sizeL; i++){
        arrL[i] = array[left + i];
    }
    for (int j = 0; j < sizeR; j++){
        arrR[j] = array[mid + 1 + j];
    }
    int i = 0, j = 0;
    int k = left;

    // Merge the temp arrays back
    while (i < sizeL && j < sizeR) {
        if (arrL[i] <= arrR[j]) {
            array[k] = arrL[i];
            i++;
        }
        else {
            array[k] = arrR[j];
            j++;
        }
        k++;
    }
    while (i < sizeL) {
        array[k] = arrL[i];
        i++;
        k++;
    }
```

```
    while (j < sizeR) {
        array[k] = arrR[j];
        j++;
        k++;
    }
}

template<typename T>
void mergeSort(T array[],int left,int right){
    if (left >= right)return;
    //calculates the midpoint
    int mid = left + (right - left) / 2;
    //recursively sorts the left and right half
    mergeSort(array, left, mid);
    mergeSort(array, mid + 1, right);
    merge(array, left, mid, right);
}
```

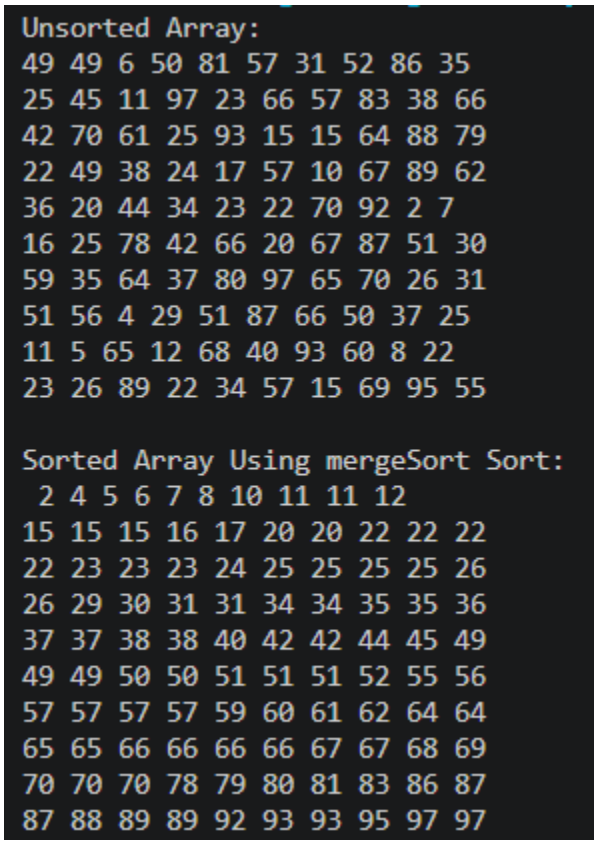
	
Observations	The merge sort algorithm is a pure divide and conquer implementation. It has a driver function that recursively isolates single elements. Inside the merge function, it is then reconstructed inside it. It splits the array exactly at the midpoint. The algorithm enforces a strict, non-overlapping binary split. The helper function instantiates local stack-allocated storage arrays (aarL and arrR) to safely implement data states. It splits datasets completely independent of the underlying values. compares these separate values, putting the lowest value in the front and the highest value in the back. It does this for both the left and right sides. After finishing both sides, it combines them back together.

Table 8-3. Merge Sort Algorithm

Code + Console Screenshot

```cpp
template<typename T>
int partition(T arr[], int low, int high) {

    // choose the pivot
    T pivot = arr[high];

    // undex of smaller element and indicates
    // the right position of pivot found so far
    int i = low - 1;

    for (int j = low; j <= high - 1; j++) {
        if (arr[j] < pivot) {
            i++;
            std::swap(arr[i], arr[j]);
        }
    }
    // move pivot after smaller elements and
    // return its position
    std::swap(arr[i + 1], arr[high]);
    return i + 1;
}
```

```cpp
template<typename T>
void quickSort(T arr[], int low, int high) {
    if (low < high) {

        // pi is the partition return index of pivot
        int pi = partition(arr, low, high);

        // recursion calls for smaller elements
        // and greater or equals elements
        quickSort(arr, low, pi - 1);
        quickSort(arr, pi + 1, high);
    }
}
```

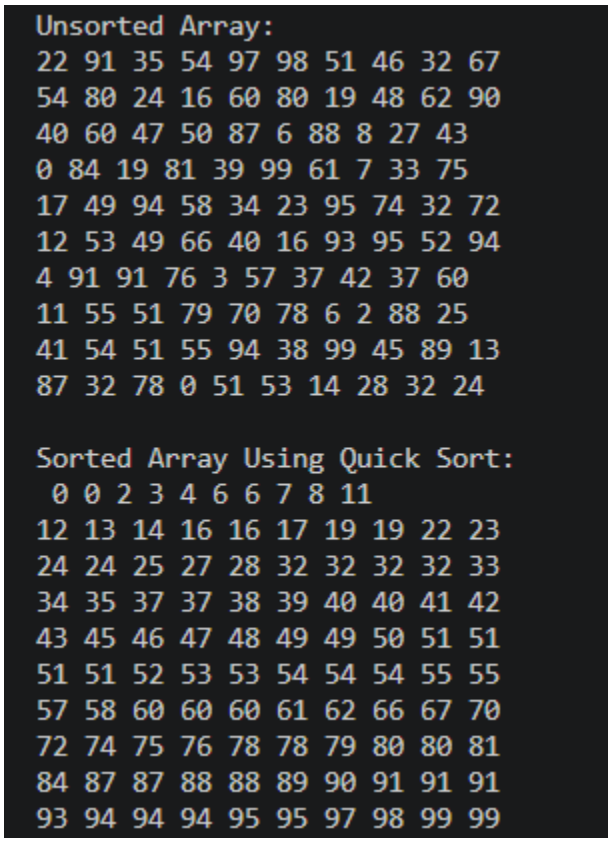
	
Observations	This sort divides the data around a single element called the “pivot”, which is placed on the far right at the last index. The algorithm checks each value in the array. If the value is greater than the pivot **(if(j > pivot))**, it just moves to the next index. If the pivot is greater than the index value **(if(j < pivot))**, it moves the temporary index to the right and swaps the values. After that, it moves the main index to the right.The last condition is, if the value is equal to the pivot **(if(j == pivot))**, the temporary index value is swapped with the pivot value. This loop continues until the sorting is complete.

Table 8-4. Quick Sort Algorithm

2. ANSWERS TO SUPPLEMENTAL ACTIVITY

Code Snippet

```cpp
#include <iostream>
#include <cstdlib>
#include <ctime>
#include <chrono>
#include "supplementary.h"

const int maxSize = 15; // Set to match your assignment's array size requirement
void displayArr(int arr[]);
void resetArray(int target[], const int source[]);

int main() {
    int originalArr[maxSize] = {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74};
    int workingArr[maxSize];

    std::cout << "=======================================\n";
    std::cout << "Original Unsorted Array:\n";
    displayArr(originalArr);
    std::cout << "=======================================\n\n";

    // --- |TESING OF SHELL SORT| ---
    resetArray(workingArr, originalArr);
    auto start = std::chrono::high_resolution_clock::now();
    shellSort(workingArr, maxSize);
    auto end = std::chrono::high_resolution_clock::now();
    std::cout << "1. Shell Sort Results:\n";
    displayArr(workingArr);
    std::cout << "Runtime: " << std::chrono::duration_cast<std::chrono::nanoseconds>(end - start).count() << " ns\n\n";

    // --- |TESTING OF MERGE SORT| ---
    resetArray(workingArr, originalArr);
    start = std::chrono::high_resolution_clock::now();
    mergeSort(workingArr, 0, maxSize - 1);
    end = std::chrono::high_resolution_clock::now();
    std::cout << "2. Merge Sort Results:\n";
    displayArr(workingArr);
    std::cout << "Runtime: " << std::chrono::duration_cast<std::chrono::nanoseconds>(end - start).count() << " ns\n\n";
```

```cpp
    // --- |TESTING OF QUICK SORT| ---
    resetArray(workingArr, originalArr);
    start = std::chrono::high_resolution_clock::now();
    quickSort(workingArr, 0, maxSize - 1);
    end = std::chrono::high_resolution_clock::now();
    std::cout << "3. Quick Sort Results:\n";
    displayArr(workingArr);
    std::cout << "Runtime: " << std::chrono::duration_cast<std::chrono::nanoseconds>(end - start).count() << " ns\n";
    std::cout << "=====================================\n";

    return 0;
}

// Helper function to print arrays
void displayArr(int arr[]) {
    for (int i = 0; i < maxSize; i++) {
        std::cout << arr[i] << " ";
        if ((i % 10) == 9) {
            std::cout << '\n';
        }
    }
    std::cout << '\n';
}

// Helper function to reset the array back to the original
//unsorted order before each sort test
void resetArray(int target[], const int source[]) {
    for (int i = 0; i < maxSize; i++) {
        target[i] = source[i];
    }
}
```

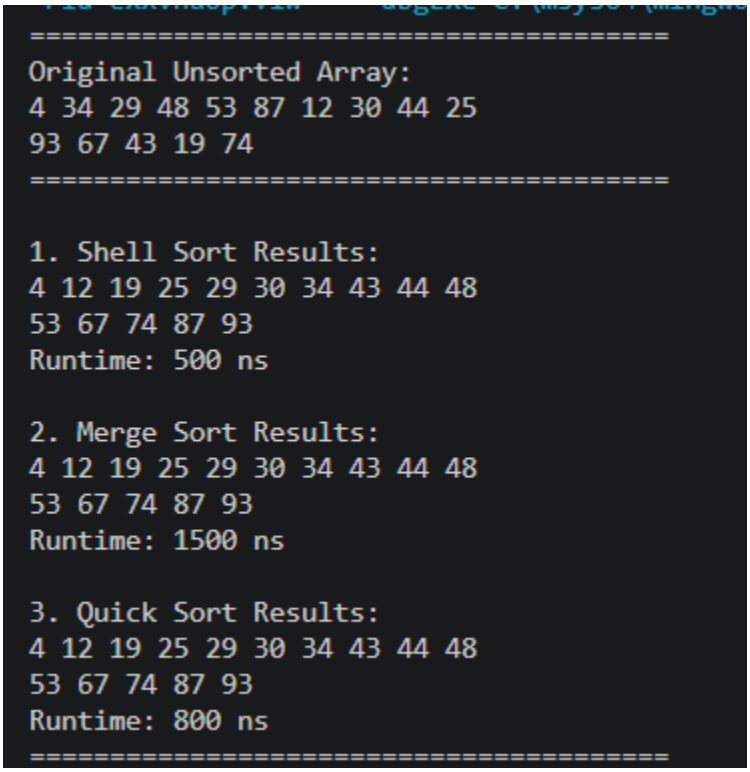
Header File

```cpp
template<typename T>
void hybridQuickShellSort(T arr[], int low, int high) {
    if (low < high) {
        //Partition the array using your quicksort partition method
        int pi = partition(arr, low, high);

        //Compute sizes of the resulting left and right segments
        int leftSize = pi - low;
        int rightSize = high - pi;

        //Delegate sorting tasks to Shell Sort instead of regular recursion
        shellSort(&arr[low], leftSize);      // Sorts the left sub-list in place
        shellSort(&arr[pi + 1], rightSize);   // Sorts the right sub-list in place
    }
}
```

Output



Problem 1: Can we sort the left sub list and right sub list from the partition method in quick sort using other sorting algorithms? Demonstrate an example.	Based on my research, it is indeed possible to sort the sub-lists from the partition step using other algorithms. I was highly introduced to hybrid algorithms such as introsort as an example of how a hybrid sorting approach works in partitioning algorithms. With the code I have analyzed and modified, combining different algorithms together, this strategy can intercept the recursion after partitioning and pass the separate memory segments like shell sort. As a reference I used AI, to also help me configure how I would approach a hybrid implementation of sorting algorithms. Which the QuickSort, the algorithm that naturally breaks a large sorting problem, Once the partition() method executes it conditions 2 structural parts, where all elements in the left sub-list are less than or equal to the pivot, and secondly, all elements in the right sub-list are greater than or equal to the pivot. We recursive self-calls of QuickSort with different algorithms like as what stated, the shell sort. Which is how the Hybrid Sorting Algorithm was made.

Problem 2: Suppose we have an array which consists of {4, 34, 29, 48, 53, 87, 12, 30, 44, 25, 93, 67, 43, 19, 74}. What sorting algorithm will give you the fastest time performance? Why can merge sort and quick sort have O(N*logN) for their time complexity?	With some help, through research definition of each sorting algorithm, and a reference approach on how to also display the time complexity of each sorting algorithm through friends, and code assistance, The fastest time performer for the given arrays is the Shell Sort. For that reason is that, it uses zero extra array spaces, which makes it exponentially faster on the pc, it is purely iterative, and because of its gap sequence, it has the ability to do wide-distance comparisons in a small array or massive indexes. As the gap decreases by, it sorts faster as it becomes small structural clusters. While research is saying QuickSort has a more efficient and optimal internal loops, it is stated that its recursive nature is a massive penalty on small lists. Why merge sort recursively breaking down a 15-element array, it is creating a heavy computational bottleneck. Which stocks down the merge sort as the lowest as it uses a lot of memory allocation. Shell sort delivers the fastest execution for this 15-element array because it scans the indices directly inside its memory boundarising using standard loop and branch predictions. Which maximizes instruction and eliminates cache misses The wide-distance gaps quickly eliminate massive inversions.

3. ARTIFICIAL INTELLIGENCE (AI) USE DISCLOSURE STATEMENT

I hereby declare that artificial intelligence (AI) tools were used, if applicable, in the preparation of this academic work. The following indicates the nature and extent of AI assistance:

Tools Used: **AI overview**

Examples: ChatGPT, Microsoft Copilot, Grammarly, QuillBot, etc.

Purpose of Use: **Code assistance explanation of concept**

Examples: grammar correction, idea generation, formatting, summarization, code assistance, explanation of concepts, etc.

Extent of Use: **Time complexity and example implementation**

Explain how AI was used and confirm that it did not replace original thinking, analysis, authorship, or completion of the required work.

By submitting this activity, I affirm that my use of AI complies with T.I.P.'s AI usage policy and does not exceed the

permitted limits for similarity, automated content generation, or academic assistance.

I understand that any false, incomplete, or misleading disclosure may be subject to investigation in accordance with due process and may result in sanctions for violation of existing T.I.P. policies.

Lab activities, templates, and related instructional materials shall not be reuploaded, reproduced, distributed, or shared with external organizations or third parties without the prior express written permission of the Technological Institute of the Philippines.